

Build the Scoped AI Template Editor

PRODUCT STATEMENT A non-technical business owner starts from a responsive template chosen by the candidate. They can edit through the canvas or code, select one element or a group, and enter a text instruction for a deterministic AI demo. A proposed change may affect only the selected elements and chosen viewport scope. Nothing changes until the user accepts it, and every manual or accepted AI change can be recovered per element without rolling back unrelated work.

1. The problem

A template editor becomes unsafe when a change can rewrite more than the user selected, break another screen size, overwrite a manual edit, or leave the user unsure about what changed. Build a credible editing loop in which elements are modular, canvas and code editing stay in sync, desktop/tablet/mobile behavior is explicit, deterministic AI proposals remain inside selection, and recovery is independent for each element.

Primary user and outcome

The primary user is a small-business owner adapting an existing website. Success means they can make substantial visual or structural changes manually, use a text-driven AI demo for selected elements, keep desktop/tablet/mobile layouts safe, approve what changes, and restore one element without disturbing the rest of the page.

Product decisions that are fixed

- Choose and load one basic, responsive, one-page template. It may be original or appropriately licensed; identify its source in the README.
- The durable source of truth is a typed, JSON-serializable template model with stable element IDs. The canvas, code editor, and AI demo must update this same state.
- Every editable element is modular. Manual editing may change content, style, position, size, order, or structure through the canvas or code surface.
- The editor provides desktop, tablet, and mobile previews. An edit can target all views or only one view; a single-view edit must leave the other views unchanged.
- Selection is authority for the AI demo: a result may reference only selected element IDs, allowed fields, and the chosen viewport scope.

- AI output is a proposal, never an automatic overwrite. In a multi-element result, the user can accept or reject and later recover each element independently.
- Every committed manual edit, accepted AI edit, and restore creates history at the element and viewport scope affected by that action.

Core capabilities

- Candidate-chosen responsive template with stable selectable elements.
- Single and multi-element selection, including click selection and either additive or drag-marquee group selection.
- High-degree manual editing through both an in-canvas surface and a code surface.
- Desktop, tablet, and mobile previews with shared edits and viewport-specific overrides.
- Text-driven deterministic AI demo with multiple documented edit paths.
- Per-element proposal review, revision history, recovery, persistence, and safe scope validation.

2. What to build

Build a browser-based Website Builder prototype around one responsive template of your choice. The editor shell and visual design are your decision; the behaviors below are required.

Required journey

1. Load: start with one basic responsive template of your choice.
2. Preview: provide desktop, tablet, and mobile views. The reviewer can switch views and see the same template resolved for that viewport.
3. Select: click one editable element to select it. Support a group through additive Shift/Ctrl/Cmd-click, drag-marquee, or both. Selection must be visible and keyboard-operable.
4. Edit on canvas: manually change content, style, position, size, order, or structure. The preview must update from the canonical template state and record a recoverable revision.
5. Edit in code: provide a code surface for the template or selected component. Valid code edits update the same state and canvas; invalid edits show an error without damaging the last valid state.

6. Choose responsive scope: allow an edit to apply to All views or one of Desktop, Tablet, or Mobile. Shared changes flow to all views unless a viewport override exists; a single-view edit must not change the other views.
7. Request an AI demo edit: select one or more elements, enter text, choose a viewport scope, and run a deterministic predefined path. The result may reference only the selected IDs and allowed fields.
8. Review and apply: show a useful before/after proposal for every returned element. Apply or reject each element independently; no template value changes before approval.
9. Recover: restore any prior manual or accepted AI revision for one element and one viewport scope without changing unrelated elements or views. Keep the recovery as a new history entry.
10. Persist: refresh preserves the current template and revision history. Include a deliberate reset action and documented deterministic demo examples.

3. Product and technical contract

Template, element and revision model

Define the schema yourself. It must be typed, serializable, and stable enough for granular canvas, code, and AI-demo updates. At minimum, model a template ID, stable element ID, element type, editable properties, base/shared values, desktop/tablet/mobile overrides, revision/version, and history by element and viewport scope. Explain the commit boundary and one trade-off in your README.

Selection and patch contract

- Selection is stored as stable element IDs, not inferred from CSS classes, text matching, or DOM position. A group remains a set of independent targets.
- Every edit command identifies its source, target IDs, viewport scope, base revision, and typed property changes. Canvas, code, and AI-demo edits use the same validation and commit pipeline.
- Base values apply across views. Desktop, tablet, or mobile overrides affect only that view. Define the resolution order clearly so one view never changes by accident.
- Unknown IDs, unselected AI targets, forbidden fields, invalid code, invalid payloads, or stale revisions must not reach current state.

- A multi-element operation may leave elements accepted, rejected, invalid, pending, or restored independently. One element's result must not force the same outcome on the others.

Deterministic AI demo

Do not connect a real model. Build a deterministic text-to-edit demo: the user enters an instruction, and a small scenario engine returns predefined typed proposals based on the instruction, selected element types, and viewport scope. The same input and state must produce the same result.

- Include reviewer-visible example instructions for: content rewrite, style change, move/resize/reorder, one-viewport responsive adjustment, and a multi-element edit.
- Include at least one safe failure example, such as an unsupported instruction, stale revision, invalid field, or unselected target.
- Each path must use the current selected IDs and values; it cannot be one fixed page replacement.
- Validate every returned proposal at runtime and render the typed template through normal React components.

Frontend quality bar

- Use React and TypeScript with clear component, state, edit-command, validation, history, responsive-resolution, and demo-engine boundaries.
- The editor must be usable at 1280 px wide. Demonstrate template previews at approximately 1440 px, 768 px, and 375 px without clipping, accidental overflow, or unusable controls.
- Selection, manual editing, viewport switching, proposal review, and restore must work with a keyboard. Form controls need labels and visible focus.
- Include focused automated tests for AI selection/field/viewport scope, canvas-code state consistency, view-specific isolation, and independent element recovery.
- Editor or component libraries are allowed. The README must identify them and show where your application owns the canonical model, validation, responsive overrides, and history.

One product decision of your own

Add one small improvement. In PRODUCT_NOTES.md, state the user problem, why you chose it, and how you would test whether it helped.

4. AI use is expected and evaluated

Use the AI coding tools you would use at work. We are evaluating how you frame tasks, provide context, inspect output, reject weak suggestions, and verify behavior—not how many prompts you send, which vendor you choose, or how much code the model writes.

Required `AI_USAGE.md`

- Tools/models used and what each was asked to help with.
- Two short, redacted examples: one planning or product-framing interaction and one implementation, debugging, or test interaction.
- One AI suggestion you rejected or materially corrected, with the reason and resulting change.
- How you checked generated code: commands/tests run, manual scenarios exercised, dependencies reviewed, and any remaining uncertainty.
- One limitation you noticed in your AI workflow and what you would change next time.

PRIVACY Do not share API keys, employer/client material, private repository content, or unrelated personal prompts. Redacted extracts are sufficient. You remain responsible for every submitted line and dependency.

Required `PRODUCT_NOTES.md`

- Primary user, job, and definition of a safe completed template edit.
- Your definition of an element, group selection, committed step, viewport scope, and editable property boundary.
- How canvas and code editing share state, and how base values resolve with desktop, tablet, and mobile overrides.
- How deterministic AI-demo paths stay inside selection and viewport scope, and how invalid or stale output is handled.
- Your review, partial-acceptance, and per-element/per-viewport recovery policy.
- The one additional capability you chose, plus the product evidence that would validate it.
- Cuts, assumptions, and the next three improvements in priority order.

5. Submission package

Deliverable	Required evidence
Repository	Accessible Git repository with meaningful commits and no secrets
Working product	Deployed link preferred; otherwise exact local setup plus a short recording
README.md	Use the supplied README template; include setup, chosen template source, demo examples, architecture and requirement mapping
AI_USAGE.md	The five evidence items above; redacted where needed
PRODUCT_NOTES.md	User, selection/context/recovery rules, one added capability, cuts and next priorities